

container orchestration and supports all cloud service providers including Amazon, Google and Microsoft Azure.

Creating a Kubernetes cluster using the Google Kubernetes Engine (GKE)

Let's suppose you have created an account on Google Cloud Platform (GCP) and also activated the billing. To create a Kubernetes cluster, first click on *Kubernetes Engine* in the compute section and select the *GKE standard* mode shown in Figure 1.

Now, enter the cluster name, its location, and specify the number of nodes and machine type. In this tutorial, the cluster has been created with the name 'osfyadityacluster', as depicted in Figure 2.

After some time, your Kubernetes cluster will be created with three node sizes, which is depicted in Figure 3.

Connecting to the Kubernetes cluster created at GKE

On your dashboard, select the newly established cluster. Click on *Connect* once you are on the information page for your cluster. You will be given a script that will set up your local computer's kubectl configuration so that you can log in and connect to your GKE cluster. Run the following command to verify that you are connected properly:

```
$ kubectl get nodes
```

This command will also give the view on pods running in the kube-system name space, as shown in Figure 4.

Frequently used commands for developing Kubernetes solutions

While developing a Kubernetes solution a user should have a good understanding about the commands to be used. Table 1 summarises the list of frequently used commands when implementing a Kubernetes cluster over the GCP. A sample screenshot of the output of `$ kubectl get nodes --output wide`

Figure 2: Create a Kubernetes cluster

Table 1: List of kubectl commands for developing Kubernetes solutions

Category	Command	Purpose
Basics	<code>\$ kubectl version</code>	Check installed version of kubectl
	<code>\$ kubectl get nodes</code>	Get Kubernetes running nodes
	<code>\$ kubectl get nodes --output wide</code>	Get nodes with IP listed
	<code>\$ kubectl get pods</code>	Get the list of running pods
	<code>\$ kubectl get all</code>	To see all created objects
	<code>\$ kubectl delete service application name</code>	Delete the running application at Kubernetes
	<code>\$ gcloud container clusters delete hello-cluster</code>	Delete the complete running cluster
Deployment	<code>\$ kubectl create f deployment name.yml</code>	Create specified name kubernetes yaml files
	<code>\$ kubectl rollout history deployment DEPLOYMENT-NAME</code>	View deployment/rollout history
	<code>\$ kubectl rollout status deployment Deployment-name</code>	Check the status of the deployment of the Deployment-name

command is shown in Figure 5.

Deploying a Node.js application on the Kubernetes cluster

The Express framework serves as the foundation for the single route endpoint in the Node.js app. The *Pod*

Manifest file will specify the public repository where the application is made available as a container image. Additionally, a service will be set up so that the application may be accessed by the general public via an outside load balancer. The *Pod Manifest* and *Service*